

NodeJs

S/N	Qestions	Answers
1	What is Node.js ?	Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside of a browser. It is built on Chrome's V8 JavaScript engine and uses an event-driven, non-blocking I/O model that makes it lightweight and efficient. Node.js is commonly used to develop web applications, but it can also be used to build other types of applications, such as command-line tools, network servers, and desktop applications.
2	Why use Node.js ?	There are many reasons why Node.js is a popular choice for developing web applications and other types of software. Here are a few of the most important reasons: • Speed and performance: Node.js is built on Chrome's V8 JavaScript engine, which is one of the fastest JavaScript engines available. This means that Node.js applications can be very fast and performant. • Scalability: Node.js is designed to be scalable. It can handle a large number of concurrent requests without blocking the main thread. This makes it a good choice for developing high-traffic web applications. • Event-driven, non-blocking I/O: Node.js uses an event-driven, non-blocking I/O model. This means that Node.js applications can handle multiple concurrent requests without blocking the main thread. This makes Node.js applications very efficient and responsive. • Cross-platform: Node.js is cross-platform, meaning that it can be run on Windows, macOS, Linux, and other operating systems. This makes it a good choice for developing applications that need to be deployed on multiple platforms. • JavaScript: Node.js is based on JavaScript, which is a popular and widely-used programming language. This makes it easy to learn and use Node.js. • Package manager: Node.js has a built-in package manager called NPM, which makes it easy to install and manage JavaScript packages. NPM has a large and growing ecosystem of packages, which can be used to develop a wide variety of applications. Besides Node. js is a good choice in developing applications such as

		• Real-time applications: Node.js is well-suited for developing real-time applications, such as chat applications and collaborative tools. This is because Node.js can handle a large number of concurrent requests without blocking the main thread. • Streaming applications: Node.js is also a good choice for developing streaming applications, such as video and audio streaming applications. This is because Node.js can efficiently handle large amounts of data. • APIs: Node.js is a good choice for developing APIs, which are used to connect different applications together. This is because Node.js is lightweight and efficient, and it can handle a large number of concurrent requests.
3	What is the event loop in Node.js	The event loop is a fundamental concept in Node.js. It is a single-threaded loop that listens for events and executes callbacks when those events occur. The event loop works by constantly checking for events in a queue. When an event is found, the event loop removes the event from the queue and executes the associated callback. The callback is a function that is passed to the event loop when the event is registered. The event loop continues to check for events and execute callbacks until there are no more events in the queue. The event loop then starts over and begins checking for events again. The event loop is what allows Node.js to handle multiple concurrent requests without blocking the main thread. When a request is received, Node.js adds the request to the event loop queue. The event loop then executes a callback when the request is completed. This allows Node.js to continue handling other requests while the current request is being processed.
4	What is the difference between synchronous and asynchronous code?	Synchronous code is code that executes in the order in which it is written. Asynchronous code is code that can execute out of order, and may not finish executing in the order in which it was written. Synchronous code is typically easier to understand and write, but it can block the main thread of execution. This can make your application unresponsive to user input or other events.
5	What is a callback function?	A callback function is a function that is passed to another function as an argument, and is executed when the other function is finished executing. Callback functions are used to implement asynchronous code, which is code that can execute out of order, and may not finish executing in the order in which it was written. Callback functions are typically used to handle the completion of asynchronous tasks, such as network requests, file reads, and

		database queries. For example, you might use a callback function to handle the completion of a network request to fetch a piece of data from an API. When the network request is complete, the callback function will be executed with the data that was fetched.
6	What are the advantages of using promises instead of callbacks?	• Readability: Promises are easier to read and write than callbacks. This is because promises are based on a more natural flow of control. • Error handling: Promises make it easier to handle errors in asynchronous code. This is because promises allow you to handle errors centrally, instead of having to handle them in multiple callback functions. • Chaining: Promises can be chained together to create complex flows of control. This can make your code more concise and easier to read.
7	What is the difference between a module and a package in Node.js?	In Node.js, a module is a single JavaScript file that has some reasonable functionality. A package is a directory with one or more modules inside of it and a package.json file which has metadata about the package. Modules are the building blocks of Node.js applications. They allow you to reuse code and organize your code into logical units. Modules can be imported into other modules, which allows you to share code between different parts of your application. Packages are a way to distribute Node.js modules. Packages can be published to the npm registry, which is a public repository of Node.js modules. You can install packages from the npm registry using the npm install command.
8	What is Express.js?	Express is a flexible Node.js web application framework that provides a set of features for developing web and mobile applications. It is designed to facilitate the rapid development of web applications and APIs.
9	What are some popular Node.js frameworks and libraries?	Nest.js, Express.js, Async, mongoose, axios, socket.io.
10	What is the difference between a single-threaded and multi-threaded language?	A single-threaded language is a programming language that only allows one thread to execute at a time. A multi-threaded language is a programming language that allows multiple threads to execute at the same time. A thread is a lightweight process that can execute independently of other threads. Threads share the same memory space as the main process, which allows them to communicate with each other easily.
11	How does Node.js handle concurrency?	Node.js is a single-threaded JavaScript runtime environment that uses an event loop to handle concurrency. This means that Node.js can

		handle multiple requests at the same time without blocking the main thread.
12	What are some best practices for writing efficient Node.js code?	Use asynchronous programming. Node.js is designed for asynchronous programming, so it is important to take advantage of this feature. Asynchronous programming allows you to handle multiple requests at the same time without blocking the main thread. Use the event loop. The event loop is the mechanism that allows Node.js to handle concurrency efficiently. When writing Node.js code, you should be aware of how the event loop works and how to use it to your advantage. Avoid blocking I/O operations. Blocking I/O operations can block the main thread and prevent Node.js from handling multiple requests at the same time. If possible, you should use non-blocking I/O operations instead. Use a caching layer. Caching can help to improve the performance of your Node.js application by storing frequently accessed data in memory. Optimize your database queries. Database queries can be a major performance bottleneck for Node.js applications. You should optimize your database queries to make them as efficient as possible. Use a profiler. A profiler can help you to identify performance bottlenecks in your Node.js application. Once you have identified the bottlenecks, you can take steps to fix them. • Use modules. Modules are a great way to organize your code and make it reusable. • Use a framework. A framework can provide you with a lot of the boilerplate code that you need to write a Node.js application. This can save you time and help you to write more efficient code. • Test your code. It is important to test your code to ensure that it is working as expected and that it is efficient. • Use a debugger. A debugger can help you to step through your code and identify errors.
13	What is caching and how can it be used in Node.js?	Caching is a technique that stores data in a temporary storage location so that future requests for that data can be served quickly. Caching can be used to improve the performance of Node.js applications by reducing the number of database queries or other expensive operations that need to be performed.
14	What is error handling in Node.js?	Error handling in Node.js is the process of handling errors that occur while your application is running. This can involve logging the error to the console, sending an email notification, or taking some other corrective action. There are a number of different ways to handle errors in Node.js. One common approach is to use the try...catch statement. The try...catch

		statement allows you to wrap a block of code in a try block and then handle any errors that occur in the catch block. Another approach to error handling in Node.js is to use the events module. The events module allows you to create custom events that can be emitted when an error occurs. You can then listen for these events and handle them accordingly. Here are some additional tips for handling errors in Node.js: • Log all errors. It is important to log all errors, even if you are able to handle them gracefully. This will help you to identify and fix bugs in your code. • Be specific. When logging errors, be as specific as possible. This will help you to debug the error and fix it more quickly. • Use a middleware. You can use a middleware to handle errors for all of the requests to your application. This can make your code more organized and easier to maintain. • Test your error handling. It is important to test your error handling code to ensure that it is working as expected.
15	What is testing and how can it be used in Node.js?	Testing is a process of evaluating the correctness or desirability of a computer program or system. It can be used to verify that the program or system meets the stated requirements or to identify any defects. There are two main types of testing: • Unit testing: Unit testing is the process of testing individual units of code, such as functions or classes. • Integration testing: Integration testing is the process of testing how different units of code work together. There are a number of different ways to test Node.js applications. One common approach is to use a testing framework, such as Jest or Mocha. Testing frameworks provide a number of features that can make testing easier and more effective, such as support for test runners, assertion libraries, and mocking.
16	What is the difference between a class and a prototype in Node.js?	A class is a blueprint for creating objects. It defines the properties and methods that all objects of that class will have. A prototype is an object that serves as a template for other objects. When you create a new object, it inherits the properties and methods of its prototype. In Node.js, all objects have a prototype. The prototype of an object is the object from which it was created.

		Classes are a newer way to define objects in JavaScript. They are similar to prototypes, but they offer some additional features, such as: • Inheritance: Classes can inherit from other classes, which allows you to reuse code and create more complex objects. • Encapsulation: Classes can encapsulate their data and methods, which makes them more secure and reusable.
17	What is the difference between a promise and an async function?	A promise is an object that represents the eventual completion (or failure) of an asynchronous operation. A promise can be in one of three states: pending, fulfilled, or rejected. An async function is a function that can be used to perform asynchronous operations. Async functions return a promise, which can be used to handle the completion (or failure) of the asynchronous operation. The main difference between a promise and an async function is that a promise is an object, while an async function is a function. This means that a promise can be passed around as a value, while an async function cannot. Another difference between promises and async functions is that promises can be chained together, while async functions cannot. This means that you can use promises to create complex asynchronous workflows.
18	What is streams and how can they be used in Node.js?	A stream is a sequence of data that is read or written over time. Streams can be used to represent data from a variety of sources, such as files, network sockets, and memory buffers. Streams are particularly useful for handling large amounts of data, or data that is not available all at once. For example, you can use a stream to read a file from disk, even if the file is larger than the amount of memory available.
19	What is clustering and how can it be used to scale Node.js applications?	Clustering is a technique for running multiple instances of a Node.js application on the same machine. This can be used to improve the performance and scalability of the application. Node.js provides a built-in cluster module that makes it easy to cluster a Node.js application. The cluster module works by creating a master process and multiple worker processes. The master process is responsible for managing the worker processes and listening for incoming requests. The worker processes are responsible for handling incoming requests and processing data. To cluster a Node.js application, you need to create a cluster script and then start the cluster using the cluster.fork() method. The cluster script should define the code that the worker processes will execute.

20	What is security and how can it be implemented in Node.js applications?	Security is the practice of protecting systems and data from unauthorized access, use, disclosure, disruption, modification, or destruction. Node.js is a popular platform for developing web applications and other types of software. However, it is important to be aware of the security risks associated with developing and deploying Node.js applications. Some of the most common security risks for Node.js applications include: Injection attacks: Injection attacks occur when an attacker injects malicious code into a Node.js application. This code can then be executed by the application and damage the system or data. Cross-site scripting (XSS) attacks: XSS attacks occur when an attacker injects malicious code into a Node.js application that is then executed by the user's browser. This code can then be used to steal the user's session cookies or other sensitive information. Cross-site request forgery (CSRF) attacks: CSRF attacks occur when an attacker tricks the user into performing an unwanted action on a Node.js application. This action can then be used to steal the user's session cookies or other sensitive information. Denial-of-service (DoS) attacks: DoS attacks attempt to overwhelm a Node.js application with traffic, making it unavailable to legitimate users. File inclusion attacks: File inclusion attacks occur when an attacker is able to include malicious files in a Node.js application. This code can then be executed by the application and damage the system or data. There are a number of things that you can do to improve the security of your Node.js applications, including: Keep your software up to date: Make sure that you are using the latest versions of Node.js and all of your dependencies. This will help to mitigate known security vulnerabilities. Use a web application firewall (WAF): A WAF can help to protect your Node.js application from a variety of attacks, including injection attacks, XSS attacks, and CSRF attacks. Use a content security policy (CSP): A CSP can help to prevent malicious code from being executed in your Node.js application. Use input validation: Validate all user input before using it in your Node.js application. This will help to prevent injection attacks and other types of attacks. Limit file access: Only allow your Node.js application to access the files that it needs. This will help to prevent file inclusion attacks. Use a security scanner: Use a security scanner to scan your Node.js application for security vulnerabilities. By implementing these security measures, you can help to protect your Node.js applications from a variety of attacks.
----	--	--

		Here are some additional tips for improving the security of Node.js applications: Use a package manager: Using a package manager will help you to keep your dependencies up to date and secure. Use security modules: There are a number of security modules available for Node.js that can help you to protect your application from attacks. Monitor your application: Monitor your application for suspicious activity and respond to any incidents promptly.
21	What is the require function and how does it work internally ?	The require() function is a built-in function in Node.js that allows you to load modules into your code. Modules are reusable pieces of code that can be used to perform common tasks, such as reading and writing files, connecting to databases, and sending emails. internally, the require() function works by searching for the module in the following locations: The current directory • The node_modules directory in the current directory • The node_modules directory in the parent directory • The node_modules directory in the global node_modules directory If the module is found in any of these locations, the require() function will load it and return its object. If the module is not found in any of these locations, the require() function will throw an error.
22	What is export and import and how does it work internally ?	Export and import are two keywords in JavaScript that allow you to share code between modules. Internally, the export and import keywords work by creating a mapping between the exported names and the exported values. When you import something into a module, the JavaScript engine looks up the exported name in the mapping and returns the exported value.

MongoDB

S/N	Question	Answers
1	What is MongoDB?	MongoDB is a document database designed for ease of development and scaling. It is a NoSQL database, which means that it does not use a traditional relational database schema. Instead, MongoDB stores data in JSON-like documents. This makes it easy to store and retrieve complex

		data structures, and it also makes MongoDB very scalable.
2	Why use MongoDB?	Flexibility: MongoDB is a document-oriented database, which means that it stores data in JSON-like documents. This makes it easy to store and retrieve complex data structures, and it also makes MongoDB very flexible. Scalability: MongoDB can be scaled horizontally by adding additional servers to the cluster. This makes it possible to handle large amounts of data and traffic. Performance: MongoDB is a very high-performance database. It can handle large amounts of data and traffic with low latency. Ease of use: MongoDB is relatively easy to learn and use, even for developers with no prior experience with NoSQL databases. Community support: MongoDB has a large and active community of users and developers. This means that there are many resources available to help you get started with MongoDB and troubleshoot any problems you encounter. MongoDB is a good choice for a wide variety of applications, including: Web applications: MongoDB is a popular choice for storing user data and content in web applications. Mobile applications: MongoDB is also a popular choice for storing data on mobile devices. Real-time data processing: MongoDB can be used to store and process real-time data, such as sensor data or social media data. Gaming: MongoDB is also used in some video games to store game state and player data. If you are looking for a database that is flexible, scalable, and performant, then MongoDB is a good option to consider. It is a popular choice for a wide variety of applications, and it has a large and active community of users and developers. Here are some additional benefits of using MongoDB: Schema-free: MongoDB does not require a predefined schema, which makes it easier to develop and evolve your applications. Rich query capabilities: MongoDB supports a variety of query operators, including text search, geospatial queries, and aggregation queries. Replication and high availability: MongoDB offers built-in replication and high availability features, which can help to ensure that your data is always available and protected.
3	What are the different types of MongoDB databases?	Sharded clusters: Sharded clusters are designed to scale horizontally by distributing data across multiple servers. This makes them ideal for applications that need to handle large amounts of data and traffic. Replica sets: Replica sets are designed to provide high availability and disaster recovery. They work by replicating data across multiple servers. If

		one server fails, the other servers can continue to serve data.
4	What is a document in MongoDB?	A document in MongoDB is a JSON-like data structure that stores data. Documents can be any size or complexity, and they can contain any type of data, including strings, numbers, arrays, and nested objects. Documents are the basic unit of data in MongoDB. They are stored in collections, which are similar to tables in a relational database. However, collections do not have a predefined schema, which means that documents in a collection can have different fields and values.
5	What is a collection in MongoDB?	A collection in MongoDB is a logical grouping of documents. It is similar to a table in a relational database, but it does not have a predefined schema. This means that documents in a collection can have different fields and values. Collections are stored in databases. A database can contain one or more collections.
6	What are some of the basic MongoDB commands?	• <code>db.collection.insertOne(document)</code>: This command inserts a single document into the specified collection. • <code>db.collection.insertMany(documents)</code>: This command inserts multiple documents into the specified collection. • <code>db.collection.find()</code>: This command returns all documents in the specified collection. • <code>db.collection.findOne()</code>: This command returns the first document in the specified collection that matches the specified query. • <code>db.collection.updateOne(query, update)</code>: This command updates the first document in the specified collection that matches the specified query. • <code>db.collection.update(query, update)</code>: This command updates all documents in the specified collection that match the specified query. • <code>db.collection.deleteOne(query)</code>: This command deletes the first document in the specified collection that matches the specified query. • <code>db.collection.deleteMany(query)</code>: This command deletes all documents in the specified collection that match the specified query.
7	What are the advantages of using MongoDB over traditional relational databases?	Flexibility: MongoDB is a document-oriented database, which means that it stores data in JSON-like documents. This makes it easy to store and retrieve complex data structures, and it also makes MongoDB very flexible. Scalability: MongoDB can be scaled horizontally by adding additional servers to the cluster. This makes it possible to handle large amounts of data and traffic. Performance: MongoDB is a very high-performance database. It can handle large amounts of data and traffic with low latency.

		Ease of use: MongoDB is relatively easy to learn and use, even for developers with no prior experience with NoSQL databases. Community support: MongoDB has a large and active community of users and developers. This means that there are many resources available to help you get started with MongoDB and troubleshoot any problems you encounter. MongoDB is a good choice for a wide variety of applications, including: Web applications: MongoDB is a popular choice for storing user data and content in web applications. Mobile applications: MongoDB is also a popular choice for storing data on mobile devices. Real-time data processing: MongoDB can be used to store and process real-time data, such as sensor data or social media data. Gaming: MongoDB is also used in some video games to store game state and player data. If you are looking for a database that is flexible, scalable, and performant, then MongoDB is a good option to consider. It is a popular choice for a wide variety of applications, and it has a large and active community of users and developers. Here are some additional benefits of using MongoDB: Schema-free: MongoDB does not require a predefined schema, which makes it easier to develop and evolve your applications. Rich query capabilities: MongoDB supports a variety of query operators, including text search, geospatial queries, and aggregation queries. Replication and high availability: MongoDB offers built-in replication and high availability features, which can help to ensure that your data is always available and protected.
8	What is the difference between a primary and a secondary replica set member?	In a MongoDB replica set, there is one primary member and one or more secondary members. The primary member is responsible for receiving and processing writes. The secondary members replicate the data from the primary member and apply it to their own datasets. If the primary member fails, one of the secondary members will be elected to become the new primary.
9	How does MongoDB handle sharding?	MongoDB sharding is a method of distributing data across multiple servers to improve scalability and performance. It is particularly well-suited for applications that need to handle large datasets or high volumes of traffic. To shard a collection in MongoDB, you must first define a shard key. The shard key is a field in the documents in the collection that will be used to distribute the data across the shards. Once you have defined a shard key, you can shard the collection using the <code>sh.shardCollection()</code> command.

		When you shard a collection, MongoDB will divide the data into chunks based on the shard key. Each chunk will be assigned to a shard in the cluster. When a client reads or writes a document in a sharded collection, the MongoDB router will determine which shard the document belongs to and forward the request to the appropriate shard.
10	What are some best practices for writing efficient MongoDB queries?	• Use indexes: Indexes are data structures that help MongoDB find documents quickly. You should create indexes on any field that you frequently query against. • Use the appropriate query operators: MongoDB supports a variety of query operators, including equality operators, inequality operators, logical operators, and regular expression operators. Choose the right query operators for your queries to improve performance. • Limit the number of results returned: If you only need a subset of the results from a query, use the limit() method to limit the number of results that are returned. This can improve performance by reducing the amount of data that needs to be transferred between the client and the server. • Use projections: Projections allow you to specify which fields you want to return from a query. This can improve performance by reducing the amount of data that needs to be transferred between the client and the server. • Use aggregation pipelines: Aggregation pipelines allow you to perform complex data transformations and analysis. Aggregation pipelines can be more efficient than traditional queries for complex data processing tasks. Here are some additional tips for writing efficient MongoDB queries: • Use the explain() method to understand how MongoDB is executing your queries. This can help you identify areas where you can improve your queries. • Monitor your query performance using the MongoDB profiler. This can help you identify slow queries and optimize them. • Keep your queries simple. Avoid using complex nested subqueries and logical expressions. • Use the \$hint() operator to force MongoDB to use a specific index for a query. This can be useful for queries that are not using the index that MongoDB would normally use.
11	What is indexing and how can it be used to improve MongoDB performance?	Indexing in MongoDB is a way to improve the performance of queries. Indexes are data structures that store copies of parts of the data in documents in a way that makes it faster to search and retrieve documents. MongoDB supports a variety of indexes, including:

		• Single-field indexes: These indexes are created on a single field in a document. • Compound indexes: These indexes are created on multiple fields in a document. • Text indexes: These indexes are used to support text search. • Geospatial indexes: These indexes are used to support geospatial queries.
12	What is MongoDB aggregation and how can it be used to perform complex data analysis?	MongoDB aggregation is a framework for performing complex data analysis on MongoDB collections. It allows you to group, summarize, and transform data to gain insights into your data. Aggregation pipelines are a series of stages that process data and pass it to the next stage in the pipeline. Each stage performs a specific operation, such as filtering, grouping, or sorting the data.
13	What is MongoDB authentication and authorization?	MongoDB authentication and authorization are security features that allow you to control who can access your database and what operations they can perform. Authentication is the process of verifying the identity of a user. MongoDB supports a variety of authentication mechanisms, including: • Username and password: This is the most common authentication mechanism. Users are required to provide a username and password to authenticate themselves. • LDAP: This authentication mechanism allows MongoDB to authenticate users against an LDAP directory server. • Kerberos: This authentication mechanism allows MongoDB to authenticate users against a Kerberos server. • X.509 certificates: This authentication mechanism allows MongoDB to authenticate users against X.509 certificates.
14	What is the difference between a wiredTiger and an in-memory storage engine?	The WiredTiger and in-memory storage engines are both available in MongoDB, but they have different strengths and weaknesses. WiredTiger is the default storage engine for MongoDB. It is a hybrid storage engine that uses both disk and memory to store data. WiredTiger is well-suited for a wide range of applications, including both read-heavy and write-heavy workloads. It also offers good performance and scalability. In-memory is a storage engine that stores data entirely in memory. This makes it the fastest storage engine available in MongoDB, but it also has some drawbacks. In-memory is not as durable as WiredTiger, and it is not suitable for workloads that require large amounts of data to be stored.

15	How does MongoDB handle transactions?	MongoDB transactions allow you to group multiple read and write operations into a single unit that is either committed or aborted together. Transactions ensure that the data in your database is always in a consistent state. Once you have started a transaction, you can execute read and write operations on documents in your database. MongoDB will track the changes that you make to the data. When you are finished with your transaction, you can either commit it or abort it. To commit a transaction, you use the <code>db.session.commitTransaction()</code> method. To abort a transaction, you use the <code>db.session.abortTransaction()</code> method. If you commit a transaction, MongoDB will make the changes that you made to the data permanent. If you abort a transaction, MongoDB will discard all of the changes that you made to the data.
16	What are some best practices for designing and deploying MongoDB applications in production?	Design Use a schema: Even though MongoDB is a schemaless database, it is still important to have a schema in mind for your data. This will help you to organize your data in a consistent way and to write more efficient queries. Normalize your data: Normalizing your data will reduce redundancy and improve the performance of your queries. Use indexes: Indexes can significantly improve the performance of your queries. Be sure to index the fields that you frequently query against. Use sharding: Sharding can improve the scalability of your MongoDB application by distributing the data across multiple servers. Deployment Use a replica set: A replica set is a group of MongoDB servers that replicate the same data. This improves the availability and durability of your data. Use a load balancer: A load balancer can distribute traffic across multiple MongoDB servers in a replica set. This improves the performance and scalability of your application. Monitor your database: It is important to monitor your MongoDB database to ensure that it is performing well and that there are no errors. You can use the MongoDB profiler to collect performance data and the MongoDB logs to troubleshoot errors. Here are some additional tips for designing and deploying MongoDB applications in production: Use a development environment: It is important to test your MongoDB application in a development environment before deploying it to production. This will help you to identify and fix any problems before they affect your

production users.

Have a rollback plan: In case of a problem, it is important to have a plan for rolling back your application to a previous version. This will minimize the impact on your production users.

Keep your MongoDB software up to date: Be sure to keep your MongoDB software up to date to the latest version. This will ensure that you have the latest bug fixes and security patches.

17 What are some common MongoDB performance problems and how can they be solved?

Problem: Slow queries

Solution:

- Use indexes to improve the performance of queries.
- Analyze your query patterns and optimize your queries accordingly.
- Use the explain() method to understand how MongoDB is executing your queries.
- Use the MongoDB profiler to identify slow queries and optimize them.

Problem: High CPU usage

Solution:

- Use indexes to reduce the amount of work that MongoDB needs to do to execute queries.
- Use a replica set to distribute the load across multiple servers.
- Tune your MongoDB server parameters.
- Upgrade to a more powerful server.

Problem: High memory usage

Solution:

- Use indexes to reduce the amount of data that MongoDB needs to load into memory.
- Tune your MongoDB server parameters.
- Upgrade to a server with more memory.

Problem: High disk I/O

Solution:

- Use indexes to reduce the amount of data that MongoDB needs to read from disk.
- Tune your MongoDB server parameters.
- Use a faster disk storage system.

		Problem: Network latency Solution: • Use a local network to connect to your MongoDB server. • Use a load balancer to distribute traffic across multiple MongoDB servers. • Tune your MongoDB server parameters. • Upgrade to a faster network connection. Problem: Connection errors Solution: • Increase the number of connections that your MongoDB server can accept. • Use a connection pool to manage your connections. • Monitor your connection logs to identify and troubleshoot connection errors.
18	What are some new features in the latest versions of MongoDB?	

ExpressJS

s/n	Questions	Answers
1	What is Express.js?	Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It is designed for building web applications and APIs, and is considered the de facto standard server framework for Node.js. Express.js is built on top of the Node.js HTTP server, and provides a number of features that make it easy to build web applications, such as:

		● Routing: Express.js allows you to define routes for your application, which are used to map incoming requests to specific handlers. ● Middleware: Express.js supports middleware, which are functions that can be used to process incoming requests and outgoing responses. Middleware can be used for a variety of purposes, such as authentication, logging, and caching. ● Templating: Express.js supports a variety of templating engines, such as Pug and EJS, which allow you to generate dynamic HTML pages.
2	Why use Express.js?	● Simplicity: Express.js is very easy to learn and use, even for developers with no prior experience with Node.js. It has a simple and intuitive API that makes it easy to get started. ● Flexibility: Express.js is a very flexible framework, and can be used to build a wide variety of web applications. It provides a number of features that make it easy to develop and deploy web applications, including routing, middleware, and templating. ● Scalability: Express.js is highly scalable, and can be used to build high-traffic web applications. It is built on top of the Node.js HTTP server, which is known for its performance and scalability. ● Performance: Express.js is very performant, and can handle a large number of concurrent requests. This makes it a good choice for building web applications that need to be able to handle a lot of traffic. ● Community support: Express.js has a large and active community, which means that there are many resources available to help you get started and troubleshoot problems. There are also many third-party libraries and modules available for Express.js, which can extend its functionality and make it even more powerful. In addition to the above reasons, here are some other benefits of using Express.js: ● Routing: Express.js provides a simple and powerful way to define routes for your application. This makes it easy to map incoming requests to specific handlers. ● Middleware: Express.js supports middleware, which are functions that can be used to process incoming requests and outgoing responses. Middleware can be used for a variety of purposes, such as authentication, logging, and caching. ● Templating: Express.js supports a variety of templating engines, such as Pug and EJS, which allow you to generate dynamic HTML pages. ● Error handling: Express.js provides built-in error handling, which makes it easy to handle errors that occur in your application. ● Testing support: Express.js is easy to test, and there are a

		number of third-party testing frameworks available for Express.js.
3	What are the benefits of using Express.js over other Node.js frameworks?	Same as number 4 answer
4	What are some of the key features of Express.js?	In addition to the above reasons, here are some other benefits of using Express.js: ● Routing: Express.js provides a simple and powerful way to define routes for your application. This makes it easy to map incoming requests to specific handlers. ● Middleware: Express.js supports middleware, which are functions that can be used to process incoming requests and outgoing responses. Middleware can be used for a variety of purposes, such as authentication, logging, and caching. ● Templating: Express.js supports a variety of templating engines, such as Pug and EJS, which allow you to generate dynamic HTML pages. ● Error handling: Express.js provides built-in error handling, which makes it easy to handle errors that occur in your application. ● Testing support: Express.js is easy to test, and there are a lot of testing frameworks supporting express
5	What is routing in Express.js?	Routing in Express.js is the process of mapping incoming HTTP requests to specific handlers. Handlers are functions that are responsible for processing requests and returning responses.
6	What is middleware in Express.js?	Middleware in Express.js is a function that has access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle. Middleware can be used to perform a variety of tasks
7	What are the different types of middleware in Express.js?	There are two main types of middleware in Express.js: ● Application-level middleware: This type of middleware is applied to all requests that are processed by Express.js. To register application-level middleware, you use the <code>app.use()</code> method. ● Router-level middleware: This type of middleware is applied to all requests that are processed by a specific router. To register router-level middleware, you use the <code>router.use()</code> method. In addition to these two main types of middleware, there are also a number of other types of middleware that can be used in Express.js, including:

		● Error handling middleware: This type of middleware is used to handle errors that occur in your application. To register error handling middleware, you use the <code>app.use()</code> method with a function that takes four parameters: <code>err</code>, <code>req</code>, <code>res</code>, and <code>next</code>. ● Built-in middleware: Express.js provides a number of built-in middleware functions, such as <code>express.static()</code>, <code>express.json()</code>, and <code>express.urlencoded()</code>. These middleware functions can be used to perform common tasks, such as serving static files, parsing JSON requests, and parsing URL-encoded requests. ● Third-party middleware: There are also a number of third-party middleware libraries available for Express.js. These libraries provide middleware functions for a variety of purposes, such as authentication, authorization, and session management.
8	How do you handle errors in Express.js?	To handle errors in Express.js, you can use error handling middleware. Error handling middleware is a function that takes four parameters: <code>err</code> , <code>req</code> , <code>res</code> , and <code>next</code> . The <code>err</code> parameter is the error object, the <code>req</code> parameter is the request object, the <code>res</code> parameter is the response object, and the <code>next</code> parameter is a function that can be called to pass control to the next middleware function in the chain.
9	How do you serve static files in Express.js?	To serve static files in Express.js, you can use the <code>express.static()</code> middleware function. The <code>express.static()</code> middleware function takes the path to the directory containing the static files as an argument. The middleware function will then serve any files in the directory at their respective URL paths.
10	How do you render templates in Express.js?	To render templates in Express.js, you can use a templating engine. A templating engine is a software application that takes a template file and generates an output file, such as an HTML file.
11	How do you use Express.js to build a REST API?	To use Express.js to build a REST API, you can follow these steps: 1. Create a new Node.js project. 2. Install the Express.js package. 3. Create a new file called <code>app.js</code>. 4. Initialize Express.js in the <code>app.js</code> file. 5. Define the routes for your API. 6. Implement the handlers for the routes. 7. Start the Express.js server.
12	How do you deploy an Express.js application to production?	Here is a general overview of the steps involved in deploying an Express.js application to production: ● Build your application. This involves running the <code>npm run build</code> command. This will create a production-ready version of your

		application. • Choose a hosting provider. If you are using a cloud hosting provider, you will need to create an account and create a new application. If you are using a dedicated server, you will need to set up the server and install the necessary software. • Deploy your application. Once you have chosen a hosting provider, you will need to deploy your application to the hosting provider. The specific steps involved will vary depending on the hosting provider that you are using. • Configure your application. Once your application is deployed, you will need to configure it. This may involve setting environment variables, configuring databases, and configuring other services. • Test your application. Once your application is configured, you should test it to make sure that it is working properly. Here are some additional tips for deploying an Express.js application to production: • Use a version control system, such as Git, to manage your code. This will make it easier to deploy your application and to roll back changes if necessary. • Use a continuous integration and continuous delivery (CI/CD) pipeline to automate the deployment process. This will save you time and effort, and it will also help to ensure that your application is deployed consistently. • Use a monitoring tool to monitor your application in production. This will help you to identify and fix problems quickly.
13	What are some best practices for writing efficient Express.js code?	1. Use middleware to handle common tasks. Middleware can be used to handle common tasks such as authentication, authorization, logging, and caching. This can improve the performance of your application by reducing the amount of code that needs to be executed for each request. 2. Use a templating engine to generate dynamic responses. Templating engines can be used to generate dynamic HTML, CSS, and JavaScript responses. This can improve the performance of your application by reducing the amount of code that needs to be sent to the client. 3. Use caching to store frequently accessed data. Caching can be used to store frequently accessed data in memory, so that it does not need to be retrieved from the database or another source for each request. This can improve the performance of your application by reducing the number of database queries that need to be made. 4. Use asynchronous programming to handle concurrent requests. Express.js supports asynchronous programming, which allows you to handle multiple requests at the same time. This can improve the performance of your application by reducing the amount of time that

		each request takes to complete. 5. Optimize your database queries. When writing database queries, it is important to optimize them so that they return the data that you need as quickly as possible. You can do this by using indexes and by writing efficient SQL queries. 6. Use a performance monitoring tool to identify performance bottlenecks. A performance monitoring tool can help you to identify performance bottlenecks in your application. This information can then be used to optimize your code and improve the performance of your application.
14	How can you use Express.js to build scalable and reliable web applications?	• Use a load balancer to distribute traffic across multiple servers. A load balancer can help to improve the performance and scalability of your application by distributing traffic across multiple servers. This can also help to improve the reliability of your application by making it less susceptible to a single server failure. • Use a database that can handle a large volume of traffic. When choosing a database for your application, it is important to choose a database that can handle a large volume of traffic. Some databases, such as MySQL and PostgreSQL, are designed to handle a large number of concurrent connections. • Cache frequently accessed data. Caching can help to improve the performance of your application by reducing the number of database queries that need to be made. You can cache data in memory, in a distributed cache, or in a database. • Use asynchronous programming to handle concurrent requests. Asynchronous programming allows you to handle multiple requests at the same time. This can improve the performance of your application by reducing the amount of time that each request takes to complete. • Use a content delivery network (CDN) to serve static assets. A CDN can help to improve the performance and reliability of your application by serving static assets, such as images, CSS, and JavaScript files, from servers that are located close to your users. • Monitor your application. It is important to monitor your application to identify and fix performance problems before they impact your users. You can use a monitoring tool to monitor the performance of your application, including the response time of your pages, the number of concurrent connections, and the CPU and memory usage of your servers.
15	What are some common security vulnerabilities in Express.js applications and how can they be mitigated?	1. Cross-Site Scripting (XSS): XSS is a vulnerability that allows attackers to inject malicious code into web pages. This code can then be executed by users who visit the web pages. To mitigate XSS, you should escape all user input before displaying it on a web

		page. You can use a library such as DOMPurify or OWASP ESAPI to help you with this. 2. SQL Injection: SQL injection is a vulnerability that allows attackers to inject malicious SQL code into database queries. This code can then be executed by the database, which can allow the attacker to read or modify data in the database. To mitigate SQL injection, you should use parameterized queries. Parameterized queries prevent user input from being directly concatenated into SQL queries. 3. Cross-Site Request Forgery (CSRF): CSRF is a vulnerability that allows attackers to force authenticated users to perform unwanted actions. To mitigate CSRF, you should use anti-CSRF tokens. Anti-CSRF tokens are unique tokens that are generated for each authenticated user. These tokens are then included in all requests that the user makes. The server verifies the anti-CSRF token on each request to ensure that it is valid. 4. Insecure Direct Object References (IDOR): IDOR is a vulnerability that allows attackers to access resources that they should not have access to. To mitigate IDOR, you should implement access control mechanisms. Access control mechanisms ensure that users can only access resources that they are authorized to access. 5. Unpatched Dependencies: Unpatched dependencies can contain security vulnerabilities. To mitigate this, you should keep your dependencies up to date. You can use a tool such as Snyk or OWASP Dependency-Check to help you with this.
16	What are some new features in the latest versions of Express.js?	

ReactJS

S/N	Questions	Answers
1	What is React?	
2	Why use React?	
3	What are the key features of React?	
4	What is the difference between a component	A component in React is a function or class that returns a React element. It encapsulates a DOM tree and can contain other components. Components can

	and an element in React?	have state and lifecycle methods. An element in React is a plain object that describes a DOM node. It cannot have state or lifecycle methods. In other words, a component is a factory for creating elements.
5	What is the difference between state and props in React?	
6	What is the lifecycle of a React component?	
7	What is JSX?	
8	What are the benefits of using JSX?	
9	What is reconciliation in react?	
10	What are some best practices for writing efficient React code?	
11	How do you handle conditional rendering in React?	
12	What is HOC?	
13	How do you manage state in React?	
14	How do you use React to build reusable components?	
15	How do you use React to build performant web applications?	
16	What are some common design patterns used in React projects?	

17	How do you use React to build server-rendered applications?	
18	How do you use React to build offline applications?	
19	What are some new features in the latest versions of React?	

Typescript

s/n	Questions	Answers
1	What is TypeScript?	TypeScript is a programming language built on JavaScript that adds static typing. This means that TypeScript can help you catch errors early in the development process, before they cause problems in your application. TypeScript is a superset of JavaScript, which means that all valid JavaScript code is also valid TypeScript code. However, TypeScript adds a number of features that make it more powerful and expressive than JavaScript
2	Why use TypeScript?	• Improved code quality: TypeScript can help you to write better code by catching errors early and by helping you to understand the types of your variables and functions. • Increased productivity: TypeScript can help you to be more productive by providing features such as code completion, refactoring, and type checking. • Better collaboration: TypeScript can help you to collaborate more effectively with other developers by making your code more readable and maintainable. • Smoother onboarding: TypeScript can help new developers to get on board with your code more quickly by making it easier for them to understand the types of your variables and functions.
3	What are the key features of TypeScript?	The key features of TypeScript are:

		• Static typing: TypeScript is a strongly typed language, which means that it can catch errors early in the development process by ensuring that types are compatible. This can help you avoid bugs such as trying to add a string to a number. • Generics: TypeScript generics allow you to write code that is more reusable and flexible. For example, you can write a function that can sort any type of array, regardless of the type of elements in the array. • Interfaces: TypeScript interfaces allow you to define the shape of objects. This can help you to write code that is more robust and easier to maintain. • Namespaces: TypeScript namespaces allow you to organize your code into modules. This can help to make your code more modular and reusable. • Decorators: TypeScript decorators allow you to modify the behavior of classes, properties, and methods. This can be useful for adding features such as dependency injection and logging.
4	What is the difference between TypeScript and JavaScript?	• Typescript is a typed system while javascript is not • Typescript catches error in compile time while javascript catches error at runtime • Typescript improves code quality while javascript project is difficult to manage large codebase • Typescript have code completion mechanism while javascript does not.
5	What is type inference?	Type inference is the ability of a compiler or interpreter to automatically deduce the types of expressions in a programming language. This can be done by analyzing the code and using various techniques, such as type checking and data flow analysis.
6	What are the different types of types in TypeScript?	• Primitive types: Primitive types are the most basic types in TypeScript. They include number, string, boolean, and any. • Literal types: Literal types are types that represent specific values. For example, the literal type 10 represents the number value 10. • Enum types: Enum types are types that represent a fixed set of values. For example, the enum type Color could represent the values red, green, and blue. • Union types: Union types are types that represent a value that can be of one of several other types. For example, the union type number string represents a value that can be either a number or a string. • Intersection types: Intersection types are types that

		represent a value that must be of all of several other types. For example, the intersection type <code>number & string</code> represents a value that must be both a number and a string. • Array types: Array types are types that represent an array of values of a specific type. For example, the array type <code>number[]</code> represents an array of numbers. • Tuple types: Tuple types are types that represent a fixed-length array of values of specific types. For example, the tuple type <code>[number, string]</code> represents an array of length 2, where the first element is a number and the second element is a string. • Object types: Object types are types that represent objects. Object types can be used to define the properties and methods of an object. For example, the object type <code>{ name: string, age: number }</code> represents an object with two properties: name and age. • Generic types: Generic types are types that can be parameterized. This allows you to write code that can be reused with different types of data. For example, the generic type <code>Array<T></code> represents an array of values of type T.
7	What is a TypeScript interface?	A TypeScript interface is a type that describes the shape of an object. It can be used to define the properties and methods of an object, but it cannot be used to create an object instance.
8	What is a TypeScript class?	A TypeScript class is a blueprint for creating objects. It contains all of the data and code that is associated with the object. Classes are used to organize code and make it more reusable and maintainable. They can also be used to protect data from being accessed or modified directly.
9	What are some best practices for writing efficient TypeScript code?	• Use type inference whenever possible. Type inference is the ability of the TypeScript compiler to automatically infer the types of variables and expressions. This can save you from having to explicitly specify the types of all of your variables and expressions, which can make your code more concise and readable. • Avoid using the any type. The any type is a special type in TypeScript that allows you to assign any value to a variable. This can be useful in some cases, but it can also lead to errors. It is generally best to avoid using the any type unless you absolutely have to. • Use interfaces and classes to define your types. Interfaces and classes can help you to define complex types that can make your code more reusable and maintainable. • Use generics to write reusable code. Generics allow you to

		write code that can be reused with different types of data. This can save you from having to write duplicate code. • Optimize your code for performance. There are a number of things you can do to optimize your TypeScript code for performance, such as avoiding unnecessary loops and using caching. Here are some additional tips for writing efficient TypeScript code: • Use TypeScript's built-in compiler options. TypeScript has a number of built-in compiler options that can help you to write more efficient code. For example, you can use the <code>--strict</code> compiler option to enable stricter type checking. • Use a linter. A linter is a tool that can help you to identify and fix potential problems in your code. There are a number of linters available for TypeScript, such as TSLint and ESLint. • Use a performance profiler. A performance profiler is a tool that can help you to identify performance bottlenecks in your code. There are a number of performance profilers available for TypeScript, such as Chrome DevTools and Node.js's built-in performance profiler.
10	How do you handle generics in TypeScript?	To use generics in TypeScript, you use angle brackets (<code><</code> and <code>></code>) to specify the types of data that your code can handle. For example, the following function takes a generic type <code>T</code> as its parameter and returns a value of type <code>T</code>:
11	How do you use TypeScript to write type-safe code?	• Use type inference whenever possible. Type inference is the ability of the TypeScript compiler to automatically infer the types of variables and expressions. This can save you from having to explicitly specify the types of all of your variables and expressions, which can make your code more concise and readable. • Avoid using the any type. The any type is a special type in TypeScript that allows you to assign any value to a variable. This can be useful in some cases, but it can also lead to errors. It is generally best to avoid using the any type unless you absolutely have to. • Use interfaces and classes to define your types. Interfaces and classes can help you to define complex types that can make your code more reusable and maintainable. • Use generics to write reusable code. Generics allow you to write code that can be reused with different types of data. This can save you from having to write duplicate code.

In addition to these general tips, there are a number of specific things you can do to write type-safe code in TypeScript. For example:

- **Use type guards to check the types of values.** Type guards are expressions that can be used to check the types of values at runtime. This can be useful for preventing errors in cases where the TypeScript compiler cannot infer the types of values.
- **Use type assertions to explicitly specify the types of values.** Type assertions allow you to tell the TypeScript compiler that a value has a certain type, even if the compiler cannot infer the type of the value. This can be useful in some cases, but it should be used with caution.
- **Use a linter.** A linter is a tool that can help you to identify and fix potential problems in your code, including type errors. There are a number of linters available for TypeScript, such as TSLint and ESLint.

By following these tips and using the features of TypeScript, you can write type-safe code that is less likely to contain errors.

Here are some additional tips for writing type-safe code in TypeScript:

- **Use named types.** Named types can help to make your code more readable and maintainable. For example, instead of using the number type, you could define a named type called `MyNumber` and use that type in your code. This can help to make it clear what kind of numbers are being used in your code.
- **Use type aliases.** Type aliases can be used to create new types based on existing types. This can be useful for creating more specific types or for renaming types. For example, you could create a type alias called `PositiveNumber` that represents the set of all positive numbers. This can help to make it clear that the type `PositiveNumber` can only be used for positive numbers.
- **Use union types.** Union types allow you to define a type that can be one of several other types. This can be useful for representing values that can have different types. For example, you could define a union type called `PersonOrCompany` that represents a value that can be either a person or a company. This can help to make it clear that the type `PersonOrCompany` can only be used for values that are either people or companies.

12	How do you use TypeScript to document your code?	TypeScript can be used to document your code in a number of ways. One way is to use TypeScript's type system to define the types of variables, functions, and classes in your code. This can help to make your code more readable and maintainable, and it can also help to catch errors early. Another way to use TypeScript to document your code is to use JSDoc comments. JSDoc comments are a special type of comment that can be used to document JavaScript and TypeScript code. JSDoc comments are supported by many IDEs and code editors, and they can be used to generate documentation for your code. To use JSDoc comments to document your code, you can add comments to your code that begin with two forward slashes and an asterisk (/**). The text of the comment will be interpreted as documentation for the code that follows the comment.
13	How do you test TypeScript code?	Typescript code can be tested by using jest for writing unit and integration test and using Cypress for end to end tests
14	What is the difference between a structural type and a nominal type?	Structural type systems focus on the shape of a type, meaning that two types are considered equivalent if they have the same properties and methods. For example, in a structural type system, the following two objects would be considered equivalent while Nominal type systems focus on the name of a type, meaning that two types are only considered equivalent if they have the same name. For example, in a nominal type system, the following two objects would not be considered equivalent
15	What is the difference between a function overload and a union type?	
16	How do you use TypeScript to decorate your code?	
17	How do you use TypeScript to write type-safe React code?	
18	What are some new features in the latest versions of TypeScript?	

Programming and Other concept

S/N	Questions	Answers
1	What is object-oriented programming (OOP)?	Object-oriented programming (OOP) is a programming paradigm that uses objects and their interactions to design applications and computer programs. OOP is a powerful way to organize code and make it more reusable, maintainable, and scalable.
2	What are the four pillars of OOP?	• Encapsulation: Encapsulation is the process of bundling data and code together into a single unit. This makes the object self-contained and easier to manage. • Abstraction: Abstraction is the process of hiding the implementation details of an object from the outside world. This makes the object more reusable and easier to understand. • Inheritance: Inheritance is the process of creating new objects based on existing objects. This allows you to reuse existing code and extend it to create new functionality. • Polymorphism: Polymorphism is the ability of objects to respond to the same message in different ways. This allows you to write more flexible and reusable code.
3	What are the benefits of using OOP?	• Modularity: OOP allows you to break down your code into smaller, more manageable modules. This makes your code easier to understand, maintain, and reuse. • Data hiding: OOP helps you to protect your data from being accessed or modified directly. This makes your code more secure and reliable. • Code reuse: OOP allows you to reuse existing code by creating new objects based on existing objects. This can save you a lot of time and effort. • Flexibility: OOP makes your code more flexible and adaptable to change. This is because you can change the implementation of an object without affecting the code that uses the object. • Scalability: OOP makes it easier to develop and maintain large and complex applications. This is because you can break down the application into smaller, more manageable objects.
4	What is a class?	A class in object-oriented programming (OOP) is a blueprint for creating objects. It contains all of the data and code that is associated with the object.
5	What is an object?	An object in object-oriented programming (OOP) is a self-contained entity that contains data and code. The data is stored in the object's attributes, and the code is stored in the object's methods. Methods are functions that are attached to objects and can be used to manipulate the object's data.
6	What is encapsulation?	

7	What is inheritance?	
8	What is polymorphism?	
9	What is abstraction?	
10	What is the difference between a class and a structure?	
11	What is the difference between a method and a function?	
12	What is the difference between an interface and an abstract class?	
13	What is multiple inheritance?	
14	What is method overloading?	
15	What is method overriding?	
16	What is dynamic dispatch?	
17	What is type safety?	
18	What is the difference between a shallow copy and a deep copy?	
19	What is the difference between a public, protected, and private member?	
20	What is the difference between a static and an instance member?	
21	What is a design pattern?	

22	What are some common OOP design patterns?	
21	What is the difference between composition and inheritance?	
22	What is dependency injection?	
21	What is a Singleton	

Function Programming

S/N	Questions	Answers
1	What is functional programming?	
2	What are the key principles of functional programming?	
3	What are the benefits of using functional programming?	
4	What is a pure function?	
5	What is a lambda expression?	
6	What is a higher-order function?	
7	What is recursion?	
8	What is the difference between a mutable and immutable variable?	
9	What is the difference between a list and an array?	
10	What is a map function?	

11	What is a filter function?	
12	What is a reduce function?	
13	What is a monad?	
14	What is currying?	
15	What is partial application?	
16	What is memoization?	
17	What is tail recursion?	
18	What is the difference between a strict and a non-strict language?	
19	What are some common functional programming design patterns?	
20		